

Programación y Algoritmos I – Tarea 9

Pedro D. Llerenas
pedro.llerenas@cimat.mx

2025-11-06

Definiciones

En este reporte, creamos el tipo de dato *Flyte*, que corresponde a la siguiente estructura de datos:

```
> struct {  
>   unsigned int sign: 1;  
>   unsigned int exponent: 4;  
>   unsigned int mantissa: 3;  
> };
```

En general, la sentencia

```
> unsigned int var: 1;
```

dentro de un struct nos define la propiedad var como un espacio en la memoria que solamente ocupa 1 bit. Entonces, en este caso, tenemos 1 bit para el signo, 4 para el exponente, y 3 para la mantisa.

Exponentes

Por construcción, la estructura tiene tamaño 8 bits = 1 byte. Para que siga los estándares de un formato de la IEEE 754, el exponente tiene las siguientes propiedades:

- $E_{\min} = 0001_2 - 0111_2 = -6$
- $E_{\max} = 1110_2 - 0111_2 = 7$
- Sesgo = $0111_2 = 7$.

Con estas definiciones, tenemos los siguientes casos:

Exponente	Mantisa = 0	Mantisa != 0
0000 ₂	0	$(-1)^{\text{sign}} \times 2^{-6} \times 0.\text{bits}$
0001 ₂ , ..., 1110 ₂	$(-1)^{\text{sign}} \times 2^{\text{exp} - 7} \times 1.0$	$(-1)^{\text{sign}} \times 2^{\text{exp} - 7} \times 1.\text{bits}$
1111 ₂	$\pm\infty$	NaN

Entonces, el número positivo más pequeño y el más grande representables por este sistema son

$$00000001_2 = 2^{-6} = 0.015625,$$

$$01110111_2 = 2^7 \times 1.875 = 240,$$

respectivamente.

Conversiones

Para realizar conversiones de decimal a *flyte*, debemos tomar en cuenta que un nivel de precisión se perderá. Por ejemplo, para convertir el número 3.42 a *flyte*, primero lo normalizamos a que tenga la forma 1.x. Notemos que

$$3 = 11_2,$$
$$0.42 = 0.84 \times 2^{-1} = 1.68 \times 2^{-2} = 3.36 \times 2^{-3} = 011_2$$

Con esto, tenemos

$$3.42 = 11.011_2 \times 2^0 = 1.1011_2 \times 2^1.$$

Como $1 = 1_2$, el exponente que usamos en la representación es $1 + 7 = 8 = 0100_2$, mientras que la mantisa es $1011_2 \rightarrow 110_2$ al redondear hacia arriba. Es decir, el número en sistema *flyte* es

$$3.42 = 00100110_2.$$

Sin embargo, para convertirlo de vuelta a decimal, notemos lo siguiente:

$$00100110_2 = 2^{8-7} \times 1.75 = 3.5,$$

es decir, hemos perdido información al representar el valor como *flyte*.

Funcionamiento del programa

Para compilar el programa, usamos

```
> g++ main.cpp -o main
```

y ejecutamos usando

```
> ./main
```

Esto nos inicia un bucle infinito que espera una operación en formato $a + b$ del usuario. Al insertar la operación, se imprimen 2 resultados:

- La operación hecha convirtiendo a y b a *flyte*, luego convirtiéndolo de vuelta a float,
- La operación hecha directamente usando `std::stof`.

Ejemplo

Por ejemplo, tenemos las siguientes interacciones

```
> Enter an operation:
> 0.23 + 1.23
> flyte operation: 0.234375 + 1.25 = 1.48438
> float operation: 0.23 + 1.23 = 1.46
>
> Enter an operation:
> 12.234 + 23.32
> flyte operation: 12 + 22 = 34
> float operation: 12.234 + 23.32 = 35.554
>
> Enter an operation:
> 42.34 / 91.25
> flyte operation: 40 / 88 = 0.454545
> float operation: 42.34 / 91.25 = 0.464
>
> Enter an operation:
> 0.23 - 1.23
> flyte operation: 0.234375 - 1.25 = -1.01562
> float operation: 0.23 - 1.23 = -1
```

```
>  
> Enter an operation:  
> 0.22 * 5  
> flyte operation: 0.21875 * 5 = 1.09375  
> float operation: 0.22 * 5 = 1.1
```

Notemos que, por construcción, la precisión se pierde a gran escala para números con magnitud mayor a 1. Esto se debe a que la misma cantidad de números se representan de 0 a 1 que de 1 a 240.

También tenemos las siguientes excepciones:

```
> Enter an operation:  
> 1 / 0  
> division by 0  
> flyte operation: 1 / 0 = inf  
> float operation: 1 / 0 = inf  
>  
> Enter an operation:  
> 0 / 0  
> division by 0  
> flyte operation: 0 / 0 = -nan  
> float operation: 0 / 0 = -nan
```